FULL STACK APPLICATION DEVELOPMENT

#SKILL - 2 Hibernate CRUD Operations

Prerequisites:

- Basic knowledge of Java
- Understanding of classes, objects, and OOP
- Familiarity with SQL basics

A retail inventory system needs to store product details such as productId, name, description, price, and quantity. The admin should be able to add new products, retrieve product information, update price or quantity, and delete discontinued products.

Your task is to implement complete CRUD operations using Hibernate with proper entity mapping using JPA annotations.

1. Create a Product entity with fields: id (primary key), name, description, price, quantity.
2. Configure Hibernate and map the Product entity to a table using JPA annotations.
3. Insert multiple Product records into the database.
4. Retrieve a product using its id.
5. Update the price or quantity of any selected product.
6. Delete a product record by id if it is discontinued.
7. Note:

a. Implement Hibernate ID generation strategies (AUTO, IDENTITY, SEQUENCE) inside the entity to observe how primary key values differ.

b. The ID can be manual or auto generated, based on your chosen strategy.

8. Push the Hibernate project into a single GitHub repository.

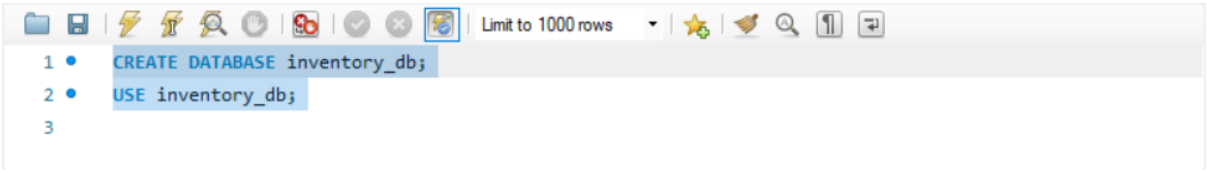
Create Database (IMPORTANT)

1. Open **MySQL Command Line Client** (or MySQL Workbench)
2. Login using password

3. Run this COMMANDS:

CREATE DATABASE inventory_db;

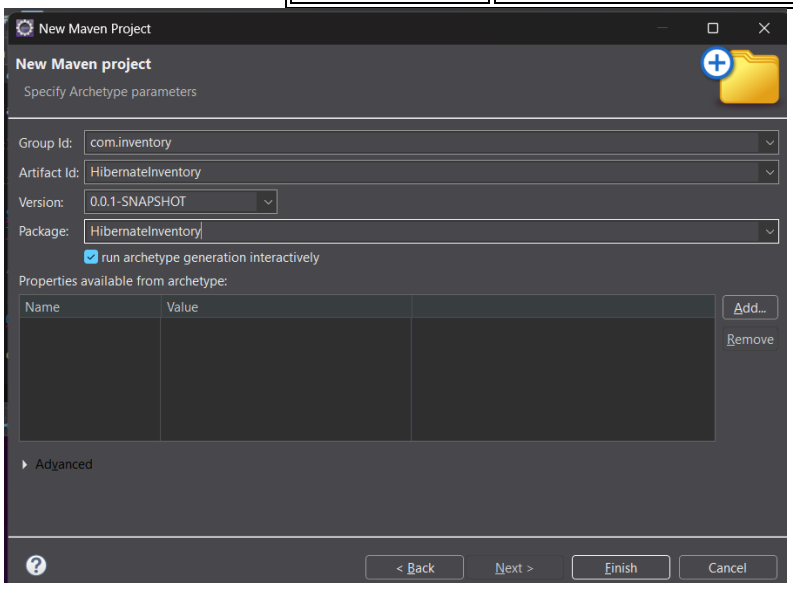
USE inventory_db;



Create Maven Project in Eclipse

1. In Eclipse menu:
   File → New → Maven Project
2. Click **Next**
3. ✅ Tick: **Create a simple project (skip archetype selection)**
4. Click **Next**

Field	Value
Group Id	com.inventory
Artifact Id	HibernateInventory
Packaging	jar
Name	HibernateInventory



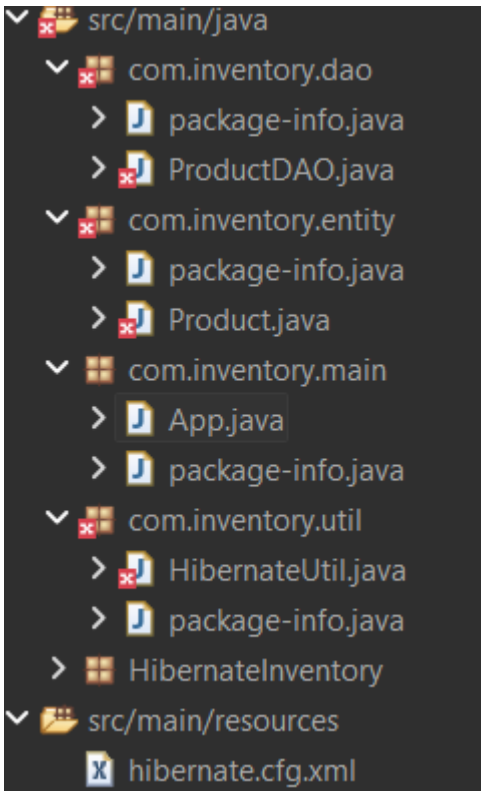
Click **Finish**

Add Maven Dependencies

1. Open **pom.xml**
2. Add this **<dependencies>**

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
</dependency>
```

Create Packages



Product.java

```
package com.inventory.entity;
import jakarta.persistence.*;
@Entity
```

```
    @Table(name = "products")
    public class Product {
      @Id
      @GeneratedValue(strategy = GenerationType.AUTO)
      private int id;
      private String name;
      private String description;
      private double price;
      private int quantity;
public Product() {}
public Product(String name, String description, double price, int quantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.quantity = quantity;
      }
      public int getId() { return id; }
      public String getName() { return name; }
      public void setName(String name) { this.name = name; }
      public String getDescription() { return description; }
      public void setDescription(String description) { this.description = description; }
      public double getPrice() { return price; }
      public void setPrice(double price) { this.price = price; }
      public int getQuantity() { return quantity; }
      public void setQuantity(int quantity) { this.quantity = quantity; }
    }
```

HibernateUtil.java

```
package com.inventory.util;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
  private static SessionFactory factory;
 static {
    factory = new Configuration().configure().buildSessionFactory();
  }
public static SessionFactory getSessionFactory() {
    return factory;
  }}
```

ProductDAO.java

```
package com.inventory.dao;
import org.hibernate.Session;
```

```
import org.hibernate.Transaction;
import com.inventory.entity.Product;
import com.inventory.util.HibernateUtil;
public class ProductDAO {
   public void saveProduct(Product product) {
      Session session = HibernateUtil.getSessionFactory().openSession();
      Transaction tx = session.beginTransaction();
      session.save(product);
      tx.commit();
      session.close();
   }
   public Product getProduct(int id) {
      Session session = HibernateUtil.getSessionFactory().openSession();
      Product product = session.get(Product.class, id);
      session.close();
      return product;
   }
   public void updateProduct(int id, double price, int quantity) {
      Session session = HibernateUtil.getSessionFactory().openSession();
      Transaction tx = session.beginTransaction();
      Product product = session.get(Product.class, id);
      product.setPrice(price);
      product.setQuantity(quantity);
      session.update(product);
      tx.commit();
      session.close();
   }
   public void deleteProduct(int id) {
      Session session = HibernateUtil.getSessionFactory().openSession();
      Transaction tx = session.beginTransaction();
 Product product = session.get(Product.class, id);
      session.delete(product);
      tx.commit();
      session.close();
   }
}
```

App.java

```
package com.inventory.main;
import com.inventory.dao.ProductDAO;
import com.inventory.entity.Product;
public class App {
   public static void main(String[] args) {
```

```
        ProductDAO dao = new ProductDAO();
       Product p1 = new Product("Laptop", "Gaming Laptop", 75000, 10);
        Product p2 = new Product("Mouse", "Wireless Mouse", 1200, 50);
       dao.saveProduct(p1);
        dao.saveProduct(p2);

        dao.updateProduct(1, 72000, 8);
        dao.deleteProduct(2);
    }
}
```

Create Hibernate Configuration File

1. Right-click src/main/resources
2. **New → File**
3. File name: hibernate.cfg.xml
4. Click **Finish**
5. Paste this:

```
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
<property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/inventory_db
        </property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">YOUR_PASSWORD</property>
 <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
 <mapping class="com.inventory.entity.Product"/>
    </session-factory>
</hibernate-configuration>
```

Note: Replace YOUR_PASSWORD

Run the Project

1. Right-click **App.java**
2. **Run As → Java Application**

Verify Output (MySQL)

SELECT * FROM products;

output

id	name	description	price	quantity
1	Laptop	Gaming Laptop	72000	8

VIVA QUESTIONS:

1. What is ORM and why is it used in application development?

A) ORM (Object Relational Mapping) is a technique that maps Java objects to database tables. It allows developers to interact with the database using Java objects instead of writing SQL queries.

Why used:

- Reduces SQL code
- Improves productivity
- Makes applications database-independent

2. Which annotation is used to mark a class as a JPA entity?

A) The @Entity annotation is used to mark a class as a JPA entity.

3. Why must every Hibernate entity have a primary key?

A) Every Hibernate entity must have a primary key to uniquely identify each record in the database and to manage persistence operations like update and delete.

4. How do you update an existing record using Hibernate?

A) To update a record, we retrieve the entity using its ID, modify its values, and then call the update() method inside a transaction.

5. What is the difference between get() and load() in Hibernate?

A)

get()	**load()**
Returns actual object	Returns proxy object
Returns null if not found	Throws exception if not found
Hits database immediately	Hits database lazily